Testing Digital Signal Processing Code without an Oracle: Metamorphic Relations, Domain Generators and Error-Model Tolerances

Clément Basdevant
Independent Researcher
ORCID 0009-0004-7496-2365

***Abstract*—Signal processing code is hard to test: the correct output of a filter, a transform or a resampler on an arbitrary input is unknown, the inputs that break such code are specific, and floating-point results can only be compared within a tolerance. A survey of five DSP libraries shows that their tests compare with typed-in values, never generate their inputs, and use hand-written tolerances chosen test by test (26 distinct relative tolerances in `scipy.signal` alone). We propose a method that derives the test from a statement of what an operator is: metamorphic relations proved for transforms, linear time-invariant filters, rational resamplers and streaming processors replace the expected output; inputs are drawn from signal families known to break DSP code; tolerances take the form of forward error bounds, with one constant per operator class instead of one value per test, a derived bound for linear recursions, and an optional calibration against extended precision. On 166 functional faults injected by mutation into four small reference implementations, it detects 90 %, fewer than the ported test suites of NumPy and SciPy (96 %); the two detect partly different faults and together detect all of them. More than half of its detections (80 of 149) come from the mutant raising an exception or hanging rather than from a violated relation. Ablations show no net gain in detection from the domain generators or the error-model tolerances: they mainly make failures appear after fewer inputs, with smaller counterexamples on hand-written faults. On the correct implementations and declarations we tested, the method raised no false alarm in 2 000 runs. Applied to SciPy, it rediscovered a known, already fixed fault (issue 22571) and found one new edge-case exception, `sosfilt` on an empty input (issue 26406, not yet confirmed by the maintainers).**

***Index Terms*—software testing, metamorphic testing, property-based testing, digital signal processing, floating-point error, test oracle**

I. Introduction

Digital signal processing (DSP) code sits at the bottom of a large software stack: the fast Fourier transform (FFT), linear filters and sample-rate converters of NumPy [1], SciPy [2] and PyTorch [3] are called by audio, communication, geophysics and machine-learning code that rarely re-checks them. A fault at this level propagates silently: a filter that loses its state on an empty block does not crash the application that streams audio through it, it makes the output diverge. Testing these operators faces three problems at once.

P1, no oracle: For most inputs nobody knows the exact output of a filter or a transform. This is the *oracle problem* of software testing [4], and it is acute for scientific software [5]. In practice, tests compare against values computed by another tool and typed into the test file, or against a second implementation that may share the same assumptions. In the suites we surveyed (Section III), 169 tests of `scipy.signal` compare with hard-coded arrays.

P2, inputs that do not exercise the faults: Hand-written tests use a few lengths and smooth or random signals. Faults in DSP code, however, hide in specific places: lengths that are prime, odd or one away from a power of two (which change the FFT algorithm), tones between two DFT bins or close to the Nyquist frequency, extreme amplitudes, and, for streaming code, empty or one-sample blocks. A concrete case: `scipy.signal.lfilter` returned an uninitialised final state when given an empty block and an initial state (SciPy issue 22571[1]), so that any block-processing loop silently diverged after an empty block, and `sosfilt` still raises on such input. Across the 2 388 tests we surveyed, only 2 combine an initial state with an empty input, and none generates its inputs.

P3, no principled tolerance: Even when the expected output is known, floating-point results are only close to it, and *how close* depends on the operation, the length and the conditioning of the problem. Tests therefore use tolerances chosen by hand: 66 distinct values in `scipy.signal`, among them 26 relative tolerances that range from 10^{-15} to 0.1 once the values below machine precision, which ask for exact equality, are set aside. A tolerance that is too loose hides a single-precision intermediate; one that is too tight produces false alarms, which teach maintainers to loosen tolerances further.

Property-based testing [6], [7] answers P2 in general by generating inputs, and metamorphic testing [8], [9] answers P1 in general by checking relations between outputs instead of the outputs themselves. Neither tells a DSP developer which relations hold for which operator, which signals to generate, nor how to compare floating-point results. And a method that claims to find faults in numerical libraries must itself be shown not to raise false alarms, since a single spurious report costs a maintainer's trust.

This article answers the three problems with one method, implemented as a tool, and evaluates it with a protocol designed to measure both its power and its soundness. The evaluation is a negative result as much as a positive one:

[1] https://github.com/scipy/scipy/issues/22571

on generated faults, the method detects fewer faults than the existing test suites, and its domain-specific components do not improve detection measurably. Its contributions are:

- *A method* that replaces the expected output by metamorphic relations proved for four classes of DSP operators (transforms, linear time-invariant filters, rational resamplers, streaming processors), draws inputs from the signal families that break DSP code, and gives each tolerance the form of a forward error bound, with one constant per operator class instead of one value per test (Section IV).
- *An evaluation protocol* that compares the method with the test suites of NumPy and SciPy on automatically generated faults, isolates the contribution of the generators and of the tolerances by ablation, and validates the method itself: false alarms on correct code, margin between observed error and tolerance, accuracy of the references against 50-digit arithmetic, and an audit of the generators (Section V).
- *A campaign on widely used libraries* that rediscovered a known fault of SciPy and found an edge-case exception, both reported to the maintainers (Section VI-D).
- *A survey* of how five DSP libraries test themselves, which grounds P1–P3 in counts rather than impressions (Section III).

The method is implemented in dsp-props, an open-source layer on top of Hypothesis; the library, the experiments and their data are available at https://github.com/clement-basdevant/dsp-props under a BSD licence.

Section II reviews related work, Section III reports the survey, Section IV describes the method, Section V the evaluation protocol and Section VI its results. Section VII discusses what the results mean for practitioners, and Section VIII concludes.

II. Background and related work

A. The oracle problem and metamorphic testing

A test oracle decides whether an output is correct. When no such procedure exists, or when it is too costly, testing faces the oracle problem [4]; scientific software is the textbook case, since it is often written precisely because the answer is not known in advance [5]. Metamorphic testing [8], [9] avoids the oracle by checking *metamorphic relations*: if f is correct, then for related inputs x and x' the outputs $f(x)$ and $f(x')$ must satisfy a known relation, e.g., $f(x + x') = f(x) + f(x')$ for a linear f. It has been applied among others to bioinformatics [10] and image processing [11]. The quality of the approach depends on the relations: a relation that holds for the faulty program too detects nothing, and finding relations is the main cost of the method. For DSP operators the relations are classical theorems (linearity, shift and convolution theorems, Parseval's identity), but which ones apply depends on what the operator is, and on finite sequences their exact form at the edges matters.

B. Property-based testing

Property-based testing (PBT), introduced by QuickCheck [6] and available in Python through Hypothesis [7], states a property over all inputs of a type, generates inputs and, when one fails, *shrinks* it to a smaller, simpler failing input [12]. PBT supplies the machinery to check metamorphic relations, but not the relations themselves, nor the generators of meaningful inputs. Interviews of PBT users identify writing properties and generators as their main difficulty [13], and Maaz et al. use language models to propose properties for Python libraries [14]. Generic generators of floating-point arrays also produce values, such as subnormal numbers, for which a purely relative tolerance is meaningless; Section VI-B shows a false alarm of this kind.

C. Self-testing of numerical programs

Blum, Luby and Rubinfeld [15] showed that a program computing a linear function can be tested, and even corrected, by checking random linear identities. Ergün [16], [17] extended the idea to multivariate linear functions such as the DFT, and Ergün, Kumar and Rubinfeld [18] to computations that are only approximately correct. The FFTW library [19] ships a self-test that implements Ergün's tester and checks linearity, impulse responses and time and frequency shifts [20]. These works establish that linearity and shift identities characterise the DFT; they do not cover filters, resamplers or streaming code, and they treat the floating-point error with a single threshold.

D. Differential testing and numerical faults

Differential testing compares several implementations of the same function [21]. It needs at least two of them, and they may share an algorithm or an assumption, e.g., the same handling of an empty block. Di Franco et al. studied 269 numerical bugs reported in NumPy, SciPy, LAPACK, GSL and Elemental [22]. They selected bugs confirmed by developers, most of them closed with a fix, and analysed 183 of those fixes: the study shows that numerical faults reach widely used, mature libraries, not that these particular faults remain in them.

E. Testing numerical and deep-learning libraries

Several tools test deep-learning libraries, whose operators overlap with DSP code. CRADLE compares the outputs of several backends of the same model [23] and Audee generates models and inputs for such comparisons [24]; both are differential testing. FreeFuzz mines API calls from open-source code [25], DeepREL infers pairs of APIs that should give related results [26], DocTer extracts input constraints from the documentation [27], and TitanFuzz generates programs with a language model [28]. These tools find crashes and inconsistencies across a whole API with little domain knowledge, but do not aim at deriving the tolerance of numerical comparisons from the operator; our method is narrower and states, for a few operator classes, which relations hold and how large their error may be. Kanewala and Bieman survey the testing of scientific software and name the oracle problem and the tolerance on floating-point results among its main difficulties [5].

F. Tolerances of test assertions

Dutta et al. found that many flaky tests in probabilistic and machine-learning projects come from assertions whose bounds do not account for randomness, and proposed FLASH, which detects them by running the tests with many seeds [29]. FLEX repairs such assertions by fitting an extreme-value distribution to the values observed over many runs and setting the bound to a chosen percentile [30], and TERA reduces the cost of stochastic regression tests by tuning their parameters [31]. FLEX and our calibration share a principle, measuring the variability of a correct program to set a bound, but FLEX targets randomness from seeds while we target rounding error, and our default tolerances come from the form of an error bound rather than from observations.

G. Floating-point error analysis

Static analysers bound or reduce the rounding error of a given expression or program: Herbie rewrites expressions to improve their accuracy [32], FPTaylor bounds round-off errors with symbolic Taylor expansions [33], and Rosa and Daisy compile and analyse programs over the reals with sound error bounds [34], [35]. Dynamic tools search for inputs that cause large errors, guided by the condition of each operation [36] or by symbolic execution [37]. These tools analyse the implementation, which our method treats as a black box; their bounds could replace our engineering constants for code they can analyse. Linear filters have their own line of work: abstract interpretation bounds the values of digital filters [38], [39], and work on fixed-point implementations evaluates the worst-case peak gain of a filter with guaranteed accuracy and uses it to choose number formats [40], [41]. The bound of Appendix B uses the same quantity.

H. Floating-point error of DSP operators

The FFT computed by a Cooley–Tukey algorithm has a worst-case relative error bounded by $O(\varepsilon \log N)$ [42], where $\varepsilon = 2^{-52}$ is the machine epsilon of double precision (`np.finfo(float).eps`), twice the unit roundoff u in which such bounds are usually stated; we use ε throughout because the library does. Its root-mean-square error on random inputs grows only as $O(\varepsilon\sqrt{\log N})$ [43]; a direct DFT, computed as N dot products of length N, has a bound that grows as $O(\varepsilon N)$ [42]. For a linear recursion, rounding errors are fed back through the poles: Appendix B derives from the standard model of floating-point arithmetic [42] a bound that grows with the ℓ_1 norm of the impulse response, which is large when poles approach the unit circle. These bounds are rarely used in tests, which fall back on fixed relative tolerances; we use them to set tolerances (Section IV-D).

I. Mutation analysis

Mutation analysis [44], [45] measures the fault-detection power of a test approach by injecting small syntactic changes, *mutants*, into a correct program and counting how many the tests detect (*kill*). Mutation operators replace an arithmetic, relational or bitwise operator, change a constant, delete a statement, and so on. Offutt et al. [46] showed that a small selective set of operators (arithmetic, relational and logical operator replacement, operand replacement and unary operator insertion) yields almost the same mutation score as the full set, which keeps mutation affordable. Two assumptions underpin the method: the *competent programmer* hypothesis, under which real faults are small deviations from a correct program, and the *coupling effect*, under which tests that kill simple faults also kill complex ones [44]. Just et al. found that mutant detection correlates with the detection of real faults in Java programs [47].

Two difficulties matter here. First, some mutants are *equivalent*: they compute the same function as the original, and no test can kill them; deciding equivalence is undecidable in general [44], so studies use heuristics. Numerical code adds a third category: a mutant may change the output only by rounding, e.g., by reordering an addition. Such a mutant is neither equivalent nor functionally wrong, and whether it should be killed depends on the accuracy the program promises. We classify mutants with an independent oracle into equivalent, accuracy and functional faults (Section V-C) and report kill rates per class, so that a false alarm on an equivalent mutant counts against an approach instead of for it. Second, first-order mutants are mostly easy to kill; we therefore also use hand-written faults modelled on real DSP bugs, including accuracy faults.

TABLE I

WHAT EACH APPROACH SUPPLIES TO THE DEVELOPER: P1, NO NEED FOR EXPECTED VALUES; P2, GENERATED INPUTS; P3, A PRINCIPLED TOLERANCE. FLEX SETS THE BOUND OF AN EXISTING ASSERTION FROM THE VALUES OBSERVED OVER MANY RUNS, WHICH ADDRESSES P3 FOR RANDOMNESS RATHER THAN ROUNDING

Approach	P1	P2	P3
Expected-value tests	–	–	–
Differential [21]	needs 2nd impl.	partly	–
DFT self-testing [16], [20]	DFT only	yes	–
PBT, MT [7], [8]	user's relations	generic	–
DL fuzzing [25], [26]	API pairs	API-wide	–
FLEX [30]	–	–	observed
This work	yes	DSP families	yes

J. Summary

Table I summarises what each family of approaches leaves to the developer. Every approach answers part of P1–P3; none supplies relations, generators and tolerances together for the operators of a DSP library, which is the gap this article addresses.

III. HOW DSP LIBRARIES TEST THEMSELVES

To ground P1–P3 in data, we analysed the test suites of `numpy.fft`, `scipy.fft`, `scipy.signal`, librosa [48] and the functional module of torchaudio, 2 388 test functions in total.

Method: A script parses every test file into a syntax tree and counts, per library: the test functions and those decorated with Hypothesis' `@given`; the numerical comparisons, i.e., calls to `assert_allclose`,

`assert_array_almost_equal`, `allclose`, `torch.testing.assert_close` and their relatives; among them, those with a hand-written tolerance, i.e., a literal passed as `rtol`, `atol`, `decimal`, `nulp` or similar, converted to a magnitude (`decimal` $= d$ means 1.5×10^{-d}); the tests that pass an empty signal, an initial filter state (`zi=`), or both; and the tests that compare with a typed-in array of at least four floating-point literals. These are syntactic heuristics: a tolerance computed at run time or an expected array loaded from a file is not counted, so the counts are lower bounds.

Three typical tests: The ported suites of Section V give a concrete picture. SciPy's `test_rank_1_IIR` filters the ramp $0, 1, \ldots, 5$ with $b = [1, -1]$, $a = [0.5, -0.5]$ and compares the output with the six typed-in values $0, 2, 4, 6, 8, 10$ to six decimals: one input, one filter, one tolerance. `test_length_factors` for `upfirdn` checks five combinations of lengths and rates against output vectors whose comment says they come from MATLAB. `test_vs_convolve` compares `upfirdn` with a straightforward implementation on 10 000 random samples, for three decimation factors, at a relative and absolute tolerance of 10^{-7}, which is a form of differential testing within the library. Several tests are regression tests added after an issue was reported, such as the two `test_short_x` tests for a one-sample input with an initial state: edge cases enter the suites after a user has met them.

No oracle (P1): Table II shows that hard-coded expected arrays are common: 220 tests in total, of which 169 in `scipy.signal`. Such arrays come from another tool, MATLAB according to the comments of some tests, or from an earlier run of the library, and they test the library against that source on a handful of inputs. The other tests compare two computations of the same quantity within the library, e.g., `lfilter` against `convolve`, which is a form of differential testing.

Few, fixed inputs (P2): No test in the five suites uses Hypothesis or any other input generator. In the tests we ported, inputs are written by hand: short ramps, a few random arrays with a fixed seed, constant signals. Edge cases are present but scattered: `scipy.signal` has 62 tests with an empty input and 19 with an initial state, but only 2 that combine both, which is the case behind the faults of Section VI-D.

Hand-written tolerances (P3): `scipy.signal` alone performs 1 645 comparisons, 584 of them with a hand-written tolerance, which take 66 distinct values. Relative and absolute tolerances must be read separately: an absolute tolerance is in the units of the data (some are in decibels), so its magnitude says nothing by itself. The 323 absolute tolerances take 37 values that we do not compare. The 240 relative tolerances take 26 values (Fig. 1); 9 of them lie below machine precision and ask for exact equality, and the others range from 10^{-15} to 0.1, 14 orders of magnitude. The most frequent are 10^{-13} (66), 10^{-7} (31), 10^{-15} (30). A relative tolerance near 10^{-15} asserts agreement to the last bits, and one near 10^{-4} would accept a single-precision computation. The survey does not tell why each value was chosen; it shows that the tolerance is set test by test rather than derived from the operation, which is P3.

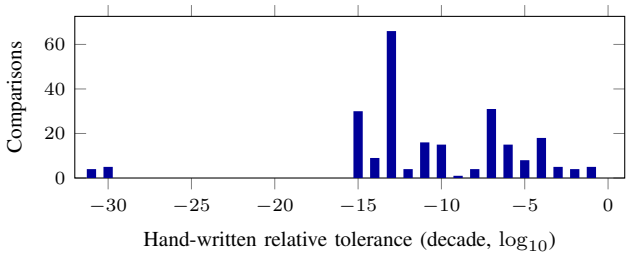


Fig. 1. Hand-written relative tolerances in the tests of `scipy.signal`, grouped by decade. Absolute tolerances are left out.

IV. Method

The method rests on one idea: the user states *what an operator is*, and everything else follows from that statement. A DFT, a causal LTI filter with a given transfer function, an L/M resampler or a streaming processor each satisfy known theorems (the relations), are known to fail on known inputs (the generators), and have a known forward error (the tolerance). The method is implemented in dsp-props as a layer on top of Hypothesis, but its ingredients do not depend on that implementation.

A. Declarations

A declaration is a decorator on the operator under test. For instance,

```
@dp.lti_filter(transfer=(b, a))
def my_filter(x):
    return scipy.signal.lfilter(b, a, x)

dp.verify(my_filter)
```

declares a causal linear time-invariant (LTI) filter with transfer function $B(z)/A(z)$. `dp.verify` runs every relation that applies to the declared class on generated inputs (200 per relation by default) and raises an error that carries the smallest failing input found by shrinking, the relation that failed, the observed error and the tolerance. The declarations available are transforms (complex DFT, DFT of real input, generic linear transforms such as the DCT), LTI filters (with an optional transfer function and gain bound), rational resamplers, and streaming or stateful processors; adapters wrap NumPy, SciPy, PyTorch and C functions called through `ctypes`.

A declaration fixes the parameters of the operator: the coefficients of a filter, the rates of a resampler. The relations then vary the input signal only. This is a deliberate choice, since parameters such as filter coefficients must satisfy constraints (stability, a declared transfer function) that a generic generator cannot know; Section VI-A shows its cost.

B. Relations (answer to P1)

Table III lists the relations for each operator class. Each is a consequence of the declaration [49], [50] and is proved in Appendix A; we state them on finite sequences of length N, as implementations compute them, which matters for shifts

TABLE II

TEST SUITES OF FIVE DSP LIBRARIES (SYNTACTIC COUNTS). TOL.: COMPARISONS WITH A HAND-WRITTEN TOLERANCE; REL.: HAND-WRITTEN RELATIVE TOLERANCES (`RTOL`, `NULP` AND SIMILAR), THEIR NUMBER OF DISTINCT VALUES AND THEIR RANGE (ABSOLUTE TOLERANCES, IN THE UNITS OF THE DATA, ARE NOT COMPARABLE ACROSS TESTS AND ARE LEFT OUT); EMPTY: TESTS WITH AN EMPTY INPUT; ZI+EMPTY: TESTS THAT ALSO PASS AN INITIAL STATE; HARD-CODED: TESTS THAT COMPARE WITH AN ARRAY OF AT LEAST FOUR TYPED-IN FLOATING-POINT VALUES

Suite	Tests	Hypothesis	Comparisons	Tol.	Rel.	Distinct	Rel. range	Empty	zi+empty	Hard-coded
NumPy (numpy.fft)	52	0	102	66	2	1	10^{-14}	1	0	1
SciPy (scipy.fft)	96	0	132	11	9	4	10^{-7}–3.0×10^{-6}	0	0	4
SciPy (scipy.signal)	1095	0	1645	584	240	26	10^{-31}–0.1	62	2	169
librosa	916	0	664	60	19	6	10^{-12}–0.01	4	0	36
torchaudio (functional)	229	0	13	12	12	6	10^{-7}–10^{-3}	0	0	10

and edges. For a transform, the shift theorem is the *circular* one, and the shift is reduced modulo N in integers so that the twiddle factors of the reference are themselves accurate. For a filter, a delay is applied by prepending zeros, and the delayed output is compared with the original output, also delayed, from zero state. For a resampler, a delay of M input samples must delay the output by exactly L samples, and the output length must be $\lceil NL/M \rceil$. For a streaming processor, the input is cut into blocks at generated positions, including empty and one-sample blocks, and the concatenated outputs must equal the one-shot output.

The relations replace the oracle: no expected output is needed, except for the optional transfer function, which the user states once instead of typing expected arrays. Without it, an LTI filter is only checked to be linear, time-invariant and causal, so a filter with the wrong coefficients but the right structure passes; this blind spot is measured in Section VI-A.

C. Generators (answer to P2)

Signals are drawn from seven families, each aimed at a known class of fault: impulses (indexing and edge errors), constants (DC gain, normalisation), sinusoids at arbitrary frequency, including off-grid ones (leakage and twiddle errors), sinusoids near the Nyquist frequency (aliasing, sign alternation), pairs of tones less than one DFT bin apart (resolution, accumulated rounding), sparse signals and white noise. Amplitudes are drawn from $\{10^{-100}, 10^{-3}, 1, 10^{3}, 10^{100}\}$ to reach underflow and overflow paths. Lengths are biased towards the values that change the code path of an FFT or a filter: 1, 2, small primes, and $2^k - 1$, 2^k, $2^k + 1$, mixed with uniform lengths up to 512. The bias is moderate: in runs of 200 inputs, the default, 15 % of the drawn lengths are among these values on average (14 % to 16 % over 20 seeds), the rest coming from the uniform branch. Block splits for streaming code mix three kinds: arbitrary cut points, which give up to $\max(N, 16)$ blocks including empty and one-sample ones, every block of a single sample, and blocks of a fixed drawn size, so that a processor that loses its state only after many calls is reached.

The families are ordered so that Hypothesis' shrinking converges to short impulses of unit amplitude, which yields counterexamples a DSP engineer can read at a glance: on the hand-written faults of Section VI-B, the median counterexample has 2 samples.

D. Tolerances (answer to P3)

All comparisons use the ℓ_2 norm of the error, scaled by the norm of a reference quantity, and a model of the admissible error. Using norms rather than element-wise comparisons follows the form of the error bounds, which are normwise, and avoids spurious failures on output samples close to zero.

FFT. For transforms,

$$\|\hat{y} - y\|_2 \le c\,\varepsilon\, \max(\log_2 N, 1)\, \|y\|_2 + N \,\mathrm{tiny}, \qquad (1)$$

the form of the forward error bound of the radix-2 FFT [42, Thm. 24.2], with a constant $c = 20$ that covers the few chained transforms in a relation, and a floor at N times the smallest normal number tiny. The floor keeps exact zeros strict while tolerating underflow on signals of amplitude 10^{-100} or subnormal values, where a purely relative tolerance would be zero. The constant is an engineering choice, not a derived value: it was set to avoid false alarms on correct libraries, and Section VI-C measures the resulting margin. Filters without a declared transfer function, resamplers and streaming processors use the same form with $c = 1000$, a looser default meant to cover implementations by direct convolution or overlap-add, whose error grows faster than $\log N$. This constant is an engineering choice too, not derived from an error analysis of these operators; the user can replace it by a calibrated model. In every case ε is the machine epsilon of the input (or of a declared precision), not of the output: an operator that returns single precision for a double-precision input is reported instead of being granted a single-precision tolerance.

Linear recursions. For a filter $y = (B/A)x$ computed by the direct form I recursion, Appendix B derives

$$\|\hat{y} - y\|_2 \le c\,\varepsilon\, k\, \|h_A\|_1 \big(\|b\|_1 + \|a_{1:}\|_1 \|h_{B/A}\|_1\big) \|x\|_2, \quad (2)$$

to first order in ε, where k is the number of coefficients other than $a_0 = 1$, h_A and $h_{B/A}$ are the impulse responses of $1/A$ and B/A, and $a_{1:}$ are the feedback coefficients. The bound grows as poles approach the unit circle, which is where recursions lose accuracy, and reduces to the error of a dot product for a FIR filter. It is the only derived tolerance of the library, and it is derived for the direct form I recursion only, to first order. Other structures, such as the transposed direct form II of `scipy.signal.lfilter`, are checked against it empirically (Section VI-C), not by proof. The factor

TABLE III
RELATIONS CHECKED FOR EACH OPERATOR CLASS. T: TRANSFORM; H: FILTER; R: RESAMPLER; D_s: DELAY BY s SAMPLES WITH ZERO FILLING; S_s: CIRCULAR SHIFT; $\omega = e^{-2\pi i/N}$

Class	Relation	Checked property
Transform	Linearity	$T(\alpha x + \beta y) = \alpha Tx + \beta Ty$
	Circular shift theorem	$T(S_s x)[k] = \omega^{ks}\, Tx[k]$
	Parseval's identity	$\|Tx\|_2^2 = c\,\|x\|_2^2$, with c declared
	Hermitian symmetry	x real $\Rightarrow Tx[N-k] = \overline{Tx[k]}$; DC and Nyquist bins real
	Inversion	$T^{-1}(Tx) = x$
	Convolution theorem	$T(x \circledast y) = Tx \cdot Ty$
LTI filter	Linearity	$H(\alpha x + \beta y) = \alpha Hx + \beta Hy$, from zero state
	Time invariance	$H(D_s x) = D_s(Hx)$
	Causality	changing $x[n]$ for $n \geq k$ leaves $y[n]$ unchanged for $n < k$
	Frequency response	DFT of the impulse response equals $B(e^{i\theta})/A(e^{i\theta})$ on the DFT grid
	Bounded gain	$\|x[n]\| \leq 1$ for all $n \Rightarrow \|y[n]\| \leq G$, with G declared
Resampler L/M	Linearity	as for filters
	Output length	$\|Rx\| = \lceil NL/M \rceil$
	Periodic time invariance	$R(D_{Ms} x) = D_{Ls}(Rx)$
	DC gain	a constant input gives the declared constant away from the edges
	Alias rejection	a tone above the output Nyquist frequency is attenuated, not folded back
Streaming	Block invariance	generated splits (Sec. IV-C), with empty and one-sample blocks, give the one-shot output

$\|h_A\|_1$ is the worst-case peak gain of $1/A$, a quantity that the literature on filter implementation evaluates rigorously [40].

Calibration. An analytic bound cannot describe every library: an FFT may switch algorithm with the factorisation of N, and its error then varies by two orders of magnitude between neighbouring lengths (Fig. 2). `dp.calibrate` measures the error of an implementation against an extended-precision reference (a DFT in `long double` where that type is wider than double, a double-double DFT elsewhere, e.g., on Windows and on macOS arm64) for every length, over several random inputs, and turns the worst error times a margin of 8 into a per-length tolerance. It refuses to calibrate when the error exceeds what rounding can explain ($10^6\varepsilon$), so that a fault cannot be absorbed into the tolerance.

E. Differences with the state of the art

Compared with generic PBT, the method supplies the three pieces of domain knowledge that PBT leaves to the user; Section VI-B measures the contribution of two of them. Compared with self-testing of the DFT [16], [20], it covers filters, resamplers and streaming code, and its tolerance follows the operator. Compared with differential testing [21], it needs a single implementation. Compared with the test suites of Section III, it needs no expected values and no tolerance chosen test by test.

V. EVALUATION PROTOCOL

We answer four research questions (RQ). **RQ1**: does the method detect more faults than the state of practice? **RQ2**: what do the domain generators and the error-model tolerances contribute? **RQ3**: is the method itself correct, i.e., free of false alarms, with sound references and generators? **RQ4**: does it find faults in real libraries? RQ1 and RQ2 use injected faults, RQ3 correct code, RQ4 the libraries themselves.

A. Subjects

We wrote four small reference implementations with the interface of a library function: a textbook radix-2 FFT with Bluestein's algorithm for other lengths; a direct-form II transposed `lfilter` with initial state; an `upfirdn` that computes each output sample directly, as a dot product of the filter with the non-zero samples of the upsampled input, which a port of `resample_poly` calls; and a block FIR filter by overlap-add. They are short enough for exhaustive mutation. Only the `lfilter` subject has the structure of its library counterpart: NumPy's FFT is a mixed-radix implementation, and SciPy's `upfirdn` is a polyphase filter bank in compiled code. The subjects therefore share the interfaces of the libraries, and so their test suites, but not their code. All four agree with NumPy or SciPy to a relative error below 1.6×10^{-15} and pass every suite below.

B. Faults

A mutation tool generates every first-order mutant of each subject with standard operators, following [44], [46]: arithmetic, bitwise and relational operator replacement, constant replacement ($k \to k \pm 1$, $c \to c + 1$), unary minus deletion, `min`/`max` swap and statement deletion. This gives 183 mutants (76 FFT, 55 `lfilter`, 36 `upfirdn`, 16 overlap-add).

We also use 31 hand-written faults in six families, modelled on real DSP bugs: a forgotten $1/N$, a twiddle sign error, single-precision intermediates, a filter that changes after 64 samples, a gain wrong by 1 %, a missing interpolation gain, a state reset between blocks. Four of them are accuracy faults (single-precision intermediates), which first-order mutation does not produce.

C. Ground truth

An oracle used only for classification runs each mutant on a fixed battery of inputs and compares it with an independent reference: all lengths up to 130 and several larger ones for the FFT; many filters, including order 0, and initial states for `lfilter`; rates up to $5/4$ and $1/7$ for `upfirdn`, with and without default arguments; random block splits for overlap-add. A mutant is *equivalent* if its largest relative error is at most 10^{-12}, an *accuracy* fault if it is between 10^{-12} and 10^{-6}, and a *functional* fault above 10^{-6}, or if it raises, returns the wrong shape or exceeds the time limit. This gives 166 functional faults and 17 equivalent mutants. No mutant falls in the accuracy band: the largest relative error of a mutant is either below 10^{-14} or above 1. First-order mutation of these subjects thus produces large faults only, and the hand-written faults cover accuracy.

D. Approaches compared

SoP (state of practice) is the test suite of the library whose interface the subject implements, ported test by test with its inputs and tolerances: NumPy's and SciPy's FFT tests, SciPy's `lfilter`, `lfilter_zi`, `upfirdn` and `resample_poly` tests, including the regression test added for issue 22571. The overlap-add subject, which has no library counterpart, gets four tests in the same style, which we wrote. *dsp-props* runs the declarations a user would write, two to four per subject, written by the author of the relations. Three ablations isolate the components: *uniform* replaces the domain generators by uniform lengths in $[1, 512]$ and values in $[-1, 1]$; *fixed-tol* replaces every tolerance model by a fixed relative tolerance of 10^{-7}, the default of `numpy.testing.assert_allclose`; *generic PBT* applies both, i.e., the same relations written with Hypothesis alone.

E. Metrics and execution

For each approach and mutant we record whether the mutant is detected, the number of generated inputs before the first failure, the length of the shrunk counterexample and the run time. Each randomised approach runs with 5 seeds and we report the mean, with the range across seeds when they disagree; SoP is deterministic and runs once. A run that exceeds 90 s counts as a detection, since a hanging test fails in continuous integration. All runs use 200 inputs per relation, the library default, in the following environment: Python 3.13.16, NumPy 2.5.3, SciPy 1.18.1, Hypothesis 6.170.2, on a 4-core Linux x86-64 machine.

VI. Results

A. RQ1: detection compared with the state of practice

Table IV gives the share of mutants detected, averaged over seeds. dsp-props detects 90 % of the 166 functional mutants and the ported suites 96 %. Neither raises an alarm on an equivalent mutant; among the ablations, only generic PBT does (Section VI-B). The two sets of detected faults are not nested: 17 mutants are detected only by the suites, 6 only by dsp-props, and together they detect 100 % of them.

Many of these detections owe little to the relations. With seed 0, dsp-props detects 149 mutants. In 69 of them a relation is violated: 56 comparisons exceed their tolerance and 13 other checks fail, such as the transfer function or the DC gain; these counts are the same in every seed. In the other 80, the mutant raises an exception, exceeds the time limit (4 mutants), or makes Hypothesis report several distinct failures at once (7 mutants, which we count here because we did not record what the individual failures were). Any test that calls the mutated code on a valid input detects most of these. The classification oracle itself raises or times out on 83 of the 166 functional mutants. On the remaining 83, which return a wrong value without raising, dsp-props detects 87 % and the suites 95 %.

Every miss of dsp-props involves operator parameters that no declaration uses. A declaration fixes the coefficients of a filter or the rates of a resampler, and dsp-props then varies only the input signal. Seven `lfilter` mutants break filters of order 0 or 1, or filters whose numerator is shorter than the denominator, and the declared filters have orders 4 and 2 with numerators at least as long. The `upfirdn` survivors change the length of the full output, which the declarations never see since they reach `upfirdn` through `resample_poly` and through a FIR filter truncated to the input length, or ignore the last input sample, which every relation sees identically on both of its sides. The suites miss other faults. Their only comparisons with expected values on the FFT use length 30, which the subject computes with Bluestein's algorithm; on powers of two they only check that `ifft` inverts `fft`, which two mutants of the radix-2 twiddle factors still satisfy. They also always pass the rates of `upfirdn` explicitly, so four mutants of its default arguments survive.

On these subjects, dsp-props detects fewer faults than the ported suites. It detects some faults that they miss, without expected values. Its misses point to the choice of declarations rather than to the relations, but we did not test whether additional declarations, such as a first-order filter, would detect them.

B. RQ2: contribution of generators and tolerances

On the generated mutants, the four randomised variants detect almost the same faults (Table IV); counting a mutant as detected when a majority of seeds detect it, they disagree on 0 functional mutants. The differences are between seeds: three FFT mutants are missed in some seeds by the variants with uniform inputs and detected in every seed by dsp-props; two of them are the twiddle-factor mutants that the ported suites miss. Generic PBT also raised a false alarm in one of its 5 seeds, on the correct FFT and on every FFT mutant equivalent to it: Hypothesis drew a subnormal input of norm 10^{-323}, for which a purely relative tolerance is zero and a rounding error of one subnormal fails Parseval's identity. The error model of dsp-props includes an absolute floor, and neither dsp-props nor its other ablations raised an alarm. Most generated mutants change the output by orders of magnitude, and almost any input reveals them. The domain generators do not make detection faster here: the median number of inputs before the

TABLE IV

MUTANTS DETECTED (%, MEAN OVER SEEDS, WITH MIN–MAX WHEN SEEDS DISAGREE). FOR EQUIVALENT MUTANTS THE FIGURE IS A FALSE-ALARM RATE AND SHOULD BE 0. SOP: PORTED TEST SUITES OF NUMPY AND SCIPY; UNIFORM: DSP-PROPS WITHOUT ITS GENERATORS; FIXED-TOL: WITHOUT ITS TOLERANCE MODELS; GENERIC PBT: WITHOUT BOTH

Subject	Class (mutants)	SoP	dsp-props	uniform	fixed-tol	generic PBT
FFT	functional (63)	97	100	98 (97–100)	100	99 (97–100)
FFT	equivalent (13)	0	0	0	0	20
`lfilter`	functional (51)	100	86	86	86	86
`lfilter`	equivalent (4)	0	0	0	0	0
`upfirdn`	functional (36)	89	72	72	72	72
overlap-add	functional (16)	100	100	100	100	100
all	functional (166)	96	90	89 (89–90)	90	89 (89–90)
all	equivalent (17)	0	0	0	0	15

TABLE V

HAND-WRITTEN MUTANTS BY FAMILY: RUNS THAT REPORT A FAILURE, OVER MUTANTS AND 5 SEEDS. A CORRECT REFERENCE MUST NEVER FAIL

Mutant	dsp-props	uniform	fixed-tol	generic PBT
FFT, reference	0/5	0/5	0/5	0/5
FFT, 8 mutants	40/40	40/40	40/40	40/40
real FFT, reference	0/5	0/5	0/5	0/5
real FFT, 5 mutants	25/25	25/25	25/25	25/25
DCT, reference	0/5	0/5	0/5	0/5
DCT, 3 mutants	15/15	15/15	15/15	15/15
IIR filter, reference	0/5	0/5	0/5	0/5
IIR filter, 5 mutants	25/25	25/25	25/25	25/25
resampler, reference	0/5	0/5	0/5	0/5
resampler, 6 mutants	30/30	30/30	29/30	30/30
streaming, reference	0/5	0/5	0/5	0/5
streaming, 4 mutants	20/20	20/20	20/20	20/20
mutants always detected	31	31	30	31
first failure, median (p90)	3 (8)	8 (20)	3 (13)	8 (27)
counterexample length, p90	19	34	57	36

first failure is 3 for dsp-props and 2 for uniform inputs (90th percentile 8 and 21), and the median counterexample has 3 samples in both cases. They cost time, with a median of 5.61 s per mutant against 3.4 s for uniform inputs and 0.001 s for a ported suite.

On the hand-written faults (Table V), every variant detects every fault in every seed, except the fixed tolerance, which misses one resampler fault in one seed. The variants differ in how fast they detect. With the domain generators, the median failure appears at input 3 (90th percentile 8); with uniform inputs at input 8 (20), and with generic PBT at input 8 (27). The 90th percentile of the counterexample length is 19 samples for dsp-props, 34 with uniform inputs, and 57 with a fixed tolerance. No variant raised a false alarm on the correct references of this table. These faults were written by the author of the relations and of the generators, which favours the generators.

In short, at 200 inputs per relation the generators and the error-model tolerances bring no net gain in detection on either set of faults. Their measurable effects are fewer inputs before a failure and shorter counterexamples on the hand-written faults, a few seed-dependent misses avoided on generated ones, and one false alarm avoided. They might matter more with a smaller budget or on faults that need a specific input; we did not measure this. One hand-written fault, a filter whose gain is wrong by 1 %, is detected only because its transfer function is declared: it is still linear, time-invariant and causal.

C. RQ3: is the method itself correct?

A reader of any fault-finding method should ask whether its alarms are real. We check four things.

Relations: Every relation is a theorem about the declared class of operators (Appendix A), so a correct implementation in exact arithmetic never violates it; only the floating-point comparison can raise a false alarm.

False alarms: We ran every declaration on correct implementations, NumPy's FFT and real FFT, SciPy's `lfilter`, `sosfilt`, `upfirdn` and `resample_poly`, and the four reference subjects, with 20 independent seeds: 2 000 runs of a relation, 248 000 floating-point comparisons, no failures. This bounds the false-alarm rate of these declarations on these implementations, in double precision on one platform; it says nothing about other operators, other parameters or other platforms, and the campaign of Section VI-D shows that a wrong declaration does raise false alarms. SciPy's `lfilter` with an initial state on empty blocks is excluded from this count because it is the known fault of issue 22571; dsp-props flags it in 5 of 5 runs on SciPy 1.18.1.

Margin: During the same runs we recorded the ratio of the observed error to the tolerance for every comparison. 63 % of the comparisons are exact (ratio 0); the 99th percentile is 0.038 and the maximum 0.21: the tolerance is 5 times larger than the worst error observed on correct code. It is nevertheless tight enough to catch the four single-precision faults (Table V).

References: Calibration and the tests of the IIR model compare against extended-precision references, a DFT computed by NumPy in `long double` and a direct recursion in `long double`. Against a 50-digit computation with mpmath [51], their relative error is at most $1.0 \times 10^{-3}\,\varepsilon$ and $5.4 \times 10^{-3}\,\varepsilon$ respectively, where ε is the double-precision machine epsilon: the references are three orders of magnitude more accurate than what they are used to measure.

Generators: We sampled each generator and checked its claims. All 19 pathological lengths are produced, impulses have exactly one non-zero sample, all signals are finite and of the requested length, and amplitudes cover 10^{-100} to 10^{100};

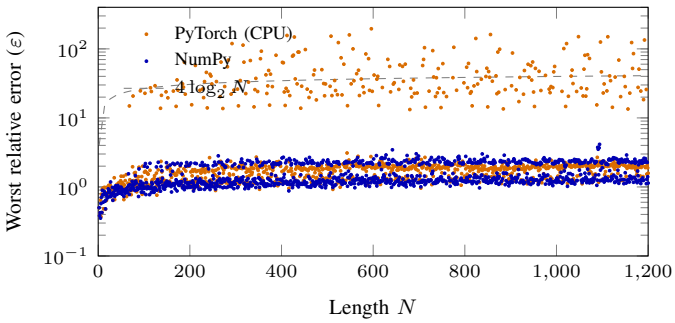


Fig. 2. Worst relative FFT error, in units of ε, versus length N, against a `long double` reference (`benchmarks/error_profile.py`). SciPy, not shown, behaves like NumPy.

66 % of block splits contain an empty block and 44 % a one-sample block; near-Nyquist frequencies lie in $[0.45, 0.5]$; 99 % of the tone pairs are less than one bin apart. The audit also found a weakness: the "off-grid" sinusoid lands exactly on a DFT bin in 41 % of draws, because Hypothesis favours simple values such as 0 and 0.5 (23 % of draws). These are useful edge cases, but the family is less often off-grid than its name says; a future version will draw both on-grid and strictly off-grid frequencies.

D. RQ4: faults in real libraries

We ran declarations of 46 operators of SciPy, NumPy, PyTorch, librosa and torchaudio. The campaign produced the following findings.

- `scipy.signal.lfilter` returns an uninitialised final state, and `sosfilt` raises, when given an empty block with an initial state. The `lfilter` part is an independent rediscovery of issue 22571, fixed in the development version after SciPy 1.18.1; the `sosfilt` part, which fails on any input that is empty along the filtered axis, was new. The same fault makes librosa's streaming pre-emphasis fail on an empty block.
- `scipy.signal.tf2sos` drops pure delays: for $y[n] = x[n-2]$ it returns an identity section, so `sosfilt(tf2sos(b, a), x)` differs from `lfilter(b, a, x)`.
- torchaudio's `deemphasis`, documented as the inverse of `preemphasis`, clamps its output to $[-1, 1]$.
- PyTorch's CPU FFT reaches $195\,\varepsilon$ for $N = 596$, against at most $4.1\,\varepsilon$ for NumPy over $N = 2, \ldots, 1200$ (Fig. 2); this is not a fault, but it shows why one fixed tolerance cannot fit every library.

The campaign also produced three alarms that triage traced to errors in our own declarations (e.g., `filtfilt` is not symmetric under time reversal with its default padding); they are listed in the repository.

E. Threats to validity

Construct. Mutants are a proxy for real faults [47]; the hand-written faults and the faults of RQ4 complement them. More than half of the detected generated mutants raise an exception or hang (Section VI-A), which any test that runs the code would notice, so the overall kill rates overstate what the relations themselves contribute; we therefore also report the rates on the mutants that do not raise. The oracle thresholds (10^{-12}, 10^{-6}) are arbitrary. No mutant falls between them, so another choice within several orders of magnitude would give the same classification, but the per-mutant errors are in the result files so that other thresholds can be applied.

Oracle extended after the first run. The classification oracle was changed once, after the approaches had been run. In the first pass it classified 158 mutants as functional and 25 as equivalent; it did not call `upfirdn` with its default arguments nor `lfilter` with an order-0 filter. The extended oracle reclassified 8 of the equivalent mutants as functional. 4 of them, all upfirdn mutants, were detected by every randomised variant in every seed and not by the ported suites, so the first pass counted them as false alarms of dsp-props; the other 4, all lfilter mutants, were detected by the suites only. The extension was motivated by the alarms on the first group: an oracle refined after seeing which approach detects what is a source of bias, even when, as here, the change moves mutants into the category where some are missed by dsp-props. With the extended oracle, no approach other than generic PBT detects any equivalent mutant.

Internal. The relations, the generators, the declarations of the comparison, the overlap-add suite and the hand-written faults were all written by the same author, which can favour the method, in particular on the hand-written faults. The declarations were not changed after the mutants were first run, but the repository history does not prove that they were fixed before any mutant was run: they were committed in the same commit as the comparison harness, shortly before the first results, and nothing records runs made before that commit. Results depend on random seeds, which we vary and report.

External. The subjects are small, and only the `lfilter` subject shares the structure of its library counterpart; the SoP suites are ports of real suites restricted to the subject's interface, and the original suites also test other data types and array shapes. The survey counts are syntactic lower bounds. The 46 targets of the campaign were chosen by hand among the public functions of the five libraries that fit a declared class or have a known identity (an inverse, a reference implementation, a block decomposition), without a sampling procedure, so the campaign shows that the method can find faults, not how often. Of these targets, three (6.5 %) raised an alarm that came from an error in our own declaration rather than from the library (Section VI-D): declarations are themselves specifications that can be wrong, and a user should expect to triage such alarms.

Reproducing: Every number and table of this article is produced by the following scripts, all in the `benchmarks/` folder of the repository:

- `survey_test_suites.py`: survey of Section III;
- `comparison/run_comparison.py`: RQ1 and RQ2, generated mutants;
- `comparison/run_hand_mutants.py`: RQ2, hand-written faults;

- `validation.py`: RQ3;
- `campaign.py` and `error_profile.py`: RQ4.

The script `paper/article/make_numbers.py` turns their result files into the numbers and tables of the article, and `proofs/check.sh` checks the Coq proofs of the appendices.

VII. Discussion

What the relations can and cannot see: A relation detects a fault only if the fault breaks it. Faults that change the operator into another operator of the same class, a filter with the wrong coefficients but still LTI and causal, a resampler that ignores the last input sample consistently, satisfy every structural relation. The transfer-function relation closes the first gap at the cost of one declaration; the second remains. Relations are thus strongest on faults of structure (state, edges, indexing, normalisation) and weakest on faults of value, where expected outputs remain useful. The tolerances leave a gap in accuracy too: a fault that degrades a transform to a few tens or hundreds of ε, such as recursively computed twiddle factors, passes the default constant, and `dp.calibrate` only refuses errors above $10^6\varepsilon$; a single-precision intermediate is caught, a slow loss of accuracy is not.

Parameters are part of the input: The misses of RQ1 all come from parameter values that no declaration exercised. A practitioner should declare an operator for several representative parameters, including degenerate ones (order 0 and 1 filters, rate 1, default arguments), much as the SoP suites do. Generating parameters as well as signals is possible for some classes, e.g., stable filters from random poles, and is the most direct way to extend the method.

Complementary, not competing: The method and expected-value tests detected overlapping but different sets of faults, and together all of them; alone, the method detected fewer. In practice they combine: a library keeps its regression tests, which pin values and parameter combinations, and adds relations, which need no values and explore inputs no one thought to write down. The cost is modest: one declaration per operator and a few seconds per run.

Soundness first: A method that reports faults in other people's code must not cry wolf. The validation of RQ3 is therefore part of the method, not an afterthought: proofs of the relations, an error model with a measured margin, references checked against 50-digit arithmetic, and generators audited against their own claims. The audit found a weakness in our own generators, and the campaign of RQ4 found three errors in our own declarations before any fault was reported upstream. Soundness here is measured, not proved: the false-alarm count covers the declarations and implementations we ran, and the relations are proved in exact arithmetic only.

VIII. Conclusion

Signal processing code is hard to test because the correct output is unknown, the inputs that break it are specific, and floating-point results can only be compared within a tolerance that nobody derives: in the libraries we surveyed, tests compare with typed-in values, never generate their inputs, and use tolerances chosen test by test, with 26 distinct relative tolerances in `scipy.signal` alone.

We proposed a method that answers each problem from a single statement of what an operator is: metamorphic relations proved for four classes of DSP operators replace the expected output, signal generators target the inputs that break DSP code, and tolerances take the form of forward error bounds, derived for linear recursions, or come from a calibration that refuses to absorb faults. We evaluated it with a protocol that measures both its power and its soundness. On 166 faults generated by mutation of four small subjects, it detects 90 %, fewer than the 96 % of the ported test suites of NumPy and SciPy; the two detect partly different faults and together detect 100 % of them. More than half of its detections come from the mutant raising an exception or hanging rather than from a violated relation, and in our ablations neither the domain generators nor the error models increased detection; they made failures appear after fewer inputs, with smaller counterexamples, on hand-written faults. Its misses involve operator parameters that no declaration exercised; the misses of the suites, code paths that their few expected values do not reach. On the correct implementations and declarations we tested, it raised no false alarm in 2 000 runs, with references and generators that we audited.

Applied to widely used libraries, the method rediscovered a known fault of SciPy's `lfilter` on empty blocks with an initial state (issue 22571, already fixed in the development version), and found that `sosfilt` raises an exception on an empty input, a new but minor edge case. It also flagged two documented behaviours that may surprise users: `tf2sos` drops pure delays, and torchaudio's `deemphasis` clamps its output. The empty-block faults are reported to SciPy[2], with a proposed fix that has not been reviewed yet[3]; the dropped delay of `tf2sos` is reported to SciPy[4] and the clamping to torchaudio[5]. None of these reports had been confirmed by a maintainer when we wrote this article.

Acknowledgments

The author thanks Nicolas Boganski for discussions during the development of dsp-props, and Thomas Calmant for feedback on the structure of this article and on its experimental protocol.

AI usage disclosure

The author conceived and directed this research, defining the problem, the scope and the design of dsp-props, choosing the experimental protocol, checking every result and submitting the bug reports to upstream projects. An AI coding assistant (Claude, Anthropic) was used under the author's direction to write code and experiment scripts and to assist with the

[2]https://github.com/scipy/scipy/issues/26406
[3]https://github.com/scipy/scipy/pull/26415
[4]https://github.com/scipy/scipy/issues/26417
[5]https://github.com/pytorch/audio/issues/4238

writing. Every number in this article is produced by a script of the repository.

Conflict of interest

The author declares no conflict of interest.

References

[1] C. R. Harris, K. J. Millman, S. J. van der Walt *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

[2] P. Virtanen, R. Gommers, T. E. Oliphant *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, 2020.

[3] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, vol. 32, 2019, pp. 8024–8035. [Online]. Available: https://papers.nips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf

[4] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015.

[5] U. Kanewala and J. M. Bieman, "Testing scientific software: A systematic literature review," *Information and Software Technology*, vol. 56, no. 10, pp. 1219–1232, 2014.

[6] K. Claessen and J. Hughes, "QuickCheck: A lightweight tool for random testing of Haskell programs," in *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP '00)*. ACM, 2000, pp. 268–279.

[7] D. R. MacIver, Z. Hatfield-Dodds, and Many Contributors, "Hypothesis: A new approach to property-based testing," *Journal of Open Source Software*, vol. 4, no. 43, p. 1891, 2019.

[8] T. Y. Chen, F.-C. Kuo, H. Liu, P.-L. Poon, D. Towey, T. H. Tse, and Z. Q. Zhou, "Metamorphic testing: A review of challenges and opportunities," *ACM Computing Surveys*, vol. 51, no. 1, pp. 4:1–4:27, 2018.

[9] S. Segura, G. Fraser, A. B. Sánchez, and A. Ruiz-Cortés, "A survey on metamorphic testing," *IEEE Transactions on Software Engineering*, vol. 42, no. 9, pp. 805–824, 2016.

[10] T. Y. Chen, J. W. K. Ho, H. Liu, and X. Xie, "An innovative approach for testing bioinformatics programs using metamorphic testing," *BMC Bioinformatics*, vol. 10, p. 24, 2009.

[11] F. Jafari and A. Nadeem, "Measuring effectiveness of metamorphic relations for image processing using mutation testing," *Journal of Imaging*, vol. 10, no. 4, p. 87, 2024.

[12] D. R. MacIver and A. F. Donaldson, "Test-case reduction via test-case generation: Insights from the Hypothesis reducer," in *34th European Conference on Object-Oriented Programming (ECOOP 2020)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), R. Hirschfeld and T. Pape, Eds., vol. 166. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, pp. 13:1–13:27.

[13] H. Goldstein, J. W. Cutler, D. Dickstein, B. C. Pierce, and A. Head, "Property-based testing in practice," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. ACM, 2024.

[14] M. Maaz, L. DeVoe, Z. Hatfield-Dodds, and N. Carlini, "Agentic property-based testing: Finding bugs across the Python ecosystem," 2025, also presented at the NeurIPS 2025 Workshop on Deep Learning for Code in the Agentic Era.

[15] M. Blum, M. Luby, and R. Rubinfeld, "Self-testing/correcting with applications to numerical problems," *Journal of Computer and System Sciences*, vol. 47, no. 3, pp. 549–595, 1993.

[16] F. Ergün, "Testing multivariate linear functions: Overcoming the generator bottleneck," in *Proceedings of the Twenty-Seventh Annual ACM Symposium on Theory of Computing (STOC '95)*. ACM, 1995, pp. 407–416.

[17] F. Ergün, S. R. Kumar, and D. Sivakumar, "Self-testing without the generator bottleneck," *SIAM Journal on Computing*, vol. 29, no. 5, pp. 1630–1651, 2000.

[18] F. Ergün, S. R. Kumar, and R. Rubinfeld, "Checking approximate computations of polynomials and functional equations," *SIAM Journal on Computing*, vol. 31, no. 2, pp. 550–576, 2001.

[19] M. Frigo and S. G. Johnson, "The design and implementation of FFTW3," *Proceedings of the IEEE*, vol. 93, no. 2, pp. 216–231, 2005.

[20] ——, "FFTW 3 source code, file `libbench2/verify-lib.c` (self-test used by `bench -verify`)," https://github.com/FFTW/fftw3/blob/master/libbench2/verify-lib.c, accessed 15 October 2026.

[21] J. Vanover, X. Deng, and C. Rubio-González, "Discovering discrepancies in numerical libraries," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '20)*. ACM, 2020, pp. 488–501.

[22] A. Di Franco, H. Guo, and C. Rubio-González, "A comprehensive study of real-world numerical bug characteristics," in *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE 2017)*. IEEE, 2017, pp. 509–519.

[23] H. V. Pham, T. Lutellier, W. Qi, and L. Tan, "CRADLE: Cross-backend validation to detect and localize bugs in deep learning libraries," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 2019, pp. 1027–1038.

[24] Q. Guo, X. Xie, Y. Li, X. Zhang, Y. Liu, X. Li, and C. Shen, "Audee: Automated testing for deep learning frameworks," in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 2020, pp. 486–498.

[25] A. Wei, Y. Deng, C. Yang, and L. Zhang, "Free lunch for testing: Fuzzing deep-learning libraries from open source," in *Proceedings of the 44th International Conference on Software Engineering (ICSE)*. ACM, 2022, pp. 995–1007.

[26] Y. Deng, C. Yang, A. Wei, and L. Zhang, "Fuzzing deep-learning libraries via automated relational API inference," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 2022, pp. 44–56.

[27] D. Xie, Y. Li, M. Kim, H. V. Pham, L. Tan, X. Zhang, and M. W. Godfrey, "DocTer: Documentation-guided fuzzing for testing deep learning API functions," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2022, pp. 176–188.

[28] Y. Deng, C. S. Xia, H. Peng, C. Yang, and L. Zhang, "Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2023, pp. 423–435.

[29] S. Dutta, A. Shi, R. Choudhary, Z. Zhang, A. Jain, and S. Misailovic, "Detecting flaky tests in probabilistic and machine learning applications," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2020, pp. 211–224.

[30] S. Dutta, A. Shi, and S. Misailovic, "FLEX: Fixing flaky tests in machine learning projects by updating assertion bounds," in *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 2021, pp. 603–614.

[31] S. Dutta, J. Selvam, A. Jain, and S. Misailovic, "TERA: Optimizing stochastic regression tests in machine learning projects," in *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2021, pp. 413–426.

[32] P. Panchekha, A. Sanchez-Stern, J. R. Wilcox, and Z. Tatlock, "Automatically improving accuracy for floating point expressions," in *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, 2015, pp. 1–11.

[33] A. Solovyev, M. S. Baranowski, I. Briggs, C. Jacobsen, Z. Rakamarić, and G. Gopalakrishnan, "Rigorous estimation of floating-point round-off errors with symbolic Taylor expansions," *ACM Transactions on Programming Languages and Systems*, vol. 41, no. 1, pp. 1–39, 2019.

[34] E. Darulova and V. Kuncak, "Sound compilation of reals," in *Proceedings of the 41st Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. ACM, 2014, pp. 235–248.

[35] E. Darulova, A. Izycheva, F. Nasir, F. Ritter, H. Becker, and R. Bastian, "Daisy – framework for analysis and optimization of numerical programs (tool paper)," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2018), Part I*, ser. Lecture Notes in Computer Science, vol. 10805. Springer, 2018, pp. 270–287.

[36] D. Zou, M. Zeng, Y. Xiong, Z. Fu, L. Zhang, and Z. Su, "Detecting floating-point errors via atomic conditions," *Proceedings of the ACM on Programming Languages*, vol. 4, no. POPL, 2020.

[37] H. Guo and C. Rubio-González, "Efficient generation of error-inducing floating-point inputs via symbolic execution," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE)*. ACM, 2020, pp. 1261–1272.
[38] J. Feret, "Static analysis of digital filters," in *Programming Languages and Systems: 13th European Symposium on Programming (ESOP 2004)*, ser. Lecture Notes in Computer Science, D. Schmidt, Ed., vol. 2986. Springer, 2004, pp. 33–48.
[39] D. Monniaux, "Compositional analysis of floating-point linear numerical filters," in *Computer Aided Verification (CAV 2005)*, ser. Lecture Notes in Computer Science. Springer, 2005, pp. 199–212.
[40] A. Volkova, T. Hilaire, and C. Lauter, "Reliable evaluation of the worst-case peak gain matrix in multiple precision," in *IEEE Symposium on Computer Arithmetic (ARITH 2015)*. IEEE, 2015, pp. 96–103.
[41] B. Lopez, T. Hilaire, and L.-S. Didier, "Formatting bits to better implement signal processing algorithms," in *Proceedings of the 4th International Conference on Pervasive and Embedded Computing and Communication Systems (PECCS)*, vol. 1, 2014, pp. 104–111.
[42] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics (SIAM), 2002.
[43] J. C. Schatzman, "Accuracy of the discrete Fourier transform and the fast Fourier transform," *SIAM Journal on Scientific Computing*, vol. 17, no. 5, pp. 1150–1166, 1996.
[44] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.
[45] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. Le Traon, and M. Harman, "Mutation testing advances: An analysis and survey," in *Advances in Computers*. Elsevier, 2019, vol. 112, pp. 275–378.
[46] A. J. Offutt, A. Lee, G. Rothermel, R. H. Untch, and C. Zapf, "An experimental determination of sufficient mutant operators," *ACM Transactions on Software Engineering and Methodology*, vol. 5, no. 2, pp. 99–118, 1996.
[47] R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, and G. Fraser, "Are mutants a valid substitute for real faults in software testing?" in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE 2014)*. ACM, 2014, pp. 654–665.
[48] B. McFee, C. Raffel, D. Liang, D. P. W. Ellis, M. McVicar, E. Battenberg, and O. Nieto, "librosa: Audio and music signal analysis in Python," in *Proceedings of the 14th Python in Science Conference (SciPy 2015)*, 2015, pp. 18–24.
[49] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Pearson, 2010.
[50] P. P. Vaidyanathan, *Multirate Systems and Filter Banks*. Englewood Cliffs, NJ: Prentice Hall, 1993.
[51] The mpmath development team, *mpmath: a Python library for arbitrary-precision floating-point arithmetic (version 1.3.0)*, 2023, main author: Fredrik Johansson. Version 1.3.0 released 2023-03-07. [Online]. Available: https://mpmath.org/

APPENDIX A
PROOFS OF THE RELATIONS

Sequences have length N, indices are taken modulo N for the DFT, and $\mathcal{F}x[k] = \sum_{n=0}^{N-1} x[n]\,\omega^{kn}$ with $\omega = e^{-2\pi i/N}$. Every proposition of this appendix is also proved in Coq, with the Mathematical Components library, in the `proofs/` directory of the repository; its README maps each proposition to its Coq theorem and lists the modelling choices. The proofs use no axiom beyond the three classical axioms of MathComp Analysis, which only the specialisation to $\omega = e^{-2\pi i/N}$ needs. Their scope is the algebra of the relations: they are stated in exact arithmetic, over fields or the reals, for idealised operators. They do not model floating-point arithmetic, do not cover the code of the library or of the operators under test, and say nothing about the tolerances, except for the exact-arithmetic steps of Appendix B.

Proposition 1 (Linearity)**.** *$\mathcal{F}$ is linear, and so is every LTI filter from zero state and every resampler of the form "upsample by L, filter, downsample by M".*

Proof. Each is a composition of linear maps: a matrix product for $\mathcal{F}$; zero insertion, convolution with h and decimation for the resampler; for a filter from zero state, y is the convolution $h * x$ truncated to N samples. □

Proposition 2 (Circular shift)**.** *If $x_s[n] = x[n-s]$, then $\mathcal{F}x_s[k] = \omega^{ks}\mathcal{F}x[k]$.*

Proof. $\sum_n x[n-s]\omega^{kn} = \sum_m x[m]\omega^{k(m+s)}$ with $m = n-s$ running over all residues. □

Proposition 3 (Parseval)**.** $\|\mathcal{F}x\|_2^2 = N\|x\|_2^2$.

Proof. $\mathcal{F} = \sqrt{N}\,U$ with U unitary, since $\sum_k \omega^{k(n-m)} = N\delta_{nm}$. □

A transform declared with another normalisation states its scale, which is checked instead of N.

Proposition 4 (Hermitian symmetry)**.** *If x is real, $\mathcal{F}x[N-k] = \overline{\mathcal{F}x[k]}$; in particular $\mathcal{F}x[0]$ and, for even N, $\mathcal{F}x[N/2]$ are real.*

Proof. $\omega^{(N-k)n} = \overline{\omega^{kn}}$ and x is real. For $k = 0$ and $k = N/2$, the bin is its own conjugate. □

Proposition 5 (Inversion and convolution)**.** *$\mathcal{F}^{-1}\mathcal{F}x = x$, and $\mathcal{F}^{-1}(\mathcal{F}x \cdot \mathcal{F}y) = x \circledast y$, the circular convolution.*

Proof. The first follows from $\mathcal{F}^{-1} = \frac{1}{N}\overline{\mathcal{F}}$ and Parseval's orthogonality. For the second, $\mathcal{F}(x \circledast y)[k] = \sum_n \sum_m x[m]y[n-m]\omega^{kn} = \mathcal{F}x[k]\,\mathcal{F}y[k]$ by the shift proposition. □

dsp-props computes $x \circledast y$ directly in $O(N^2)$, independently of the operator under test.

Proposition 6 (Time invariance and causality on finite sequences)**.** *Let f be a causal LTI filter applied from zero state, $f(x)[n] = \sum_{m \le n} h[n-m]x[m]$. Then (i) $f([0_s, x]) = [0_s, f(x)]$ on $N+s$ samples, and (ii) if x and x' agree on $[0, k)$, then $f(x)$ and $f(x')$ agree on $[0, k)$.*

Proof. (i) For $n < s$ the sum only involves zeros; for $n \ge s$, substitute $m' = m - s$. (ii) $f(x)[n]$ for $n < k$ only involves $x[m]$, $m \le n < k$. □

For a non-causal filter (e.g., a centred FIR), (i) holds after discarding the first s output samples, which is what the library checks.

Proposition 7 (Transfer function)**.** *For a causal filter with transfer function $H = B/A$ whose impulse response h is absolutely summable, and k with $A(e^{2\pi ik/L}) \neq 0$, the length-L DFT of $(h[0], \ldots, h[L-1])$ equals $H(e^{2\pi ik/L})$ up to the aliasing term $\sum_{n \ge L} h[n]\omega_L^{kn}$, whose modulus is at most $\sum_{n \ge L} |h[n]|$.*

Proof. $H(e^{i\theta}) = \sum_{n \ge 0} h[n]e^{-i\theta n}$ converges absolutely for a stable filter; split the sum at L. □

The library chooses L so that $r^L \approx 10^{-17}$, with r the largest pole modulus, capped at 2^{16} samples. Since the cap leaves a large tail when r is close to 1 ($r^L \approx 1.4 \times 10^{-3}$ for $r = 0.9999$), the tolerance includes a bound on $\sum_{n \geq L} |h[n]|$ rather than assuming it small: beyond the numerator, h follows the recursion $s_{n+1} = Fs_n$ with F the companion matrix of A, and for $r < \rho < 1$ the solution P of $F^\mathsf{T} P F - \rho^2 P = -I$ gives $\sum_{n \geq L} |h[n]| \leq \|c\|_{P^{-1}} \|s_L\|_P / (1 - \rho)$, with $c = -a_{1:}/a_0$, whatever the multiplicity of the poles.

Proposition 8 (Bounded gain)**.** *For an LTI filter with $\|h\|_1 \leq G$, every input with $|x[n]| \leq 1$ gives $|y[n]| \leq G$, and the input $x[m] = \operatorname{sign}(h[N-1-m])$ attains $\sum_{n<N} |h[n]|$ at $n = N-1$.*

Proof. $|y[n]| \leq \sum_m |h[n-m]||x[m]| \leq \|h\|_1$; with the given input, every term of $y[N-1] = \sum_m h[N-1-m]x[m]$ is $|h[N-1-m]|$. □

Proposition 9 (Periodic time invariance)**.** *Let $L, M \geq 1$ and $f = \downarrow_M \circ (h * \cdot) \circ \uparrow_L$, the filter being applied from zero state. Delaying the input by Ms samples delays the output by Ls samples.*

Proof. $\uparrow_L$ turns a delay of Ms into a delay of LMs; the filter preserves it; $\downarrow_M$ turns a delay of LMs, a multiple of M, into a delay of Ls. □

The output length $\lceil NL/M \rceil$, the DC gain and the alias rejection are declared specifications rather than theorems; their tolerances (ripple, stop-band level) are part of the declaration.

Proposition 10 (Block invariance)**.** *Let a streaming processor be a state machine $(q_{t+1}, y_t) = \phi(q_t, x_t)$ applied sample by sample, and let* `process` *on a block apply ϕ to its samples in order, starting from the current state and storing the final one. Then for any split $x = x^{(1)} \cdots x^{(J)}$, including empty blocks,* concat_j `process`$(x^{(j)})$ = `process`(x) *from the same initial state.*

Proof. By induction on J: an empty block leaves the state unchanged and outputs nothing, and processing $x^{(1)}$ then $x^{(2)}$ applies ϕ to the samples of $x^{(1)}x^{(2)}$ in order. □

The proposition holds for every split; the library checks only the splits that its generator draws: arbitrary cut points giving up to $\max(N, 16)$ blocks, including empty and one-sample ones, every block of a single sample, or blocks of one drawn size (Section IV-C). Block-based algorithms such as overlap-add are not sample-by-sample machines, but a correct one computes the same function, which is what the relation checks.

Appendix B
Error bound of a linear recursion

Normalise $a_0 = 1$. The direct form I recursion computes $y[n] = \sum_{j=0}^{q} b_j x[n-j] - \sum_{j=1}^{p} a_j y[n-j]$ with $k = p+q+1$ products, one per coefficient other than a_0. The bound below is derived for this structure only; other structures, such as the transposed direct form II, have other error propagation paths.

Units. Error analyses state the standard model with the unit roundoff u: $\mathrm{fl}(a \operatorname{op} b) = (a \operatorname{op} b)(1+\theta)$ with $|\theta| \leq u$, and $u = 2^{-53}$ in double precision with rounding to nearest [42]. The library, like NumPy, uses the machine epsilon $\varepsilon =$ `np.finfo(float).eps` $= 2^{-52} = 2u$. The derivation and the Coq proofs below are written with u; we convert to ε at the end.

Derivation. With the standard model and Higham's inner-product bound [42], the computed $\hat{y}$ satisfies $\hat{y}[n] = \sum_j b_j x[n-j] - \sum_j a_j \hat{y}[n-j] + \delta[n]$ with $|\delta[n]| \leq \gamma_k \big(\sum_j |b_j||x[n-j]| + \sum_j |a_j||\hat{y}[n-j]| \big)$, $\gamma_k = ku/(1-ku)$. The Coq proof takes this bound on δ as a hypothesis, for any $\gamma_k \geq 0$; the standard model itself is not formalised. Everything after it is proved. The error $e = \hat{y} - y$ obeys $A(z)e = \delta$, i.e., $e = h_A * \delta$. By Young's inequality $\|e\|_2 \leq \|h_A\|_1 \|\delta\|_2$, and $\|\delta\|_2 \leq \gamma_k (\|b\|_1 \|x\|_2 + \|a_{1:}\|_1 \|\hat{y}\|_2)$. Finally $\|\hat{y}\|_2 \leq \|h_{B/A}\|_1 \|x\|_2 + \|e\|_2$, which gives, to first order and with $\gamma_k \approx ku$,

$$\|\hat{y} - y\|_2 \leq u\, k\, \|h_A\|_1 \big(\|b\|_1 + \|a_{1:}\|_1 \|h_{B/A}\|_1 \big) \|x\|_2 .$$

Rigorously, if $ku \leq 1/2$ and $2ku\|h_A\|_1 \|a_{1:}\|_1 \leq 1/2$, the same bound holds with $4u$ in place of u for one computed output against the exact one.

Constant of the library. In units of $\varepsilon = 2u$, the first-order bound is (2) with $c = 1/2$ and the rigorous one with $c = 2$. A relation compares computed outputs that each carry their own error, and passes as reference norm the sum of the norms of their inputs, so the difference it checks is bounded by twice these values: $c = 1$ to first order, $c = 4$ rigorously. The library uses $c = 4$, i.e., the rigorous bound for the difference, under the two smallness conditions above; it ignores the few roundings of the relation's own arithmetic, such as the scaling of the outputs in the linearity relation. The validation of Section VI-C confirms that the bound is never reached on correct code, including `scipy.signal.lfilter`, whose transposed direct form II the derivation does not cover.

The factor $\|h_A\|_1$ is the worst-case peak gain (WCPG) of $1/A$, which bounds the amplification of any bounded perturbation injected into the recursion. The literature on the implementation of linear filters evaluates the WCPG with guaranteed accuracy and uses it to bound rounding errors and choose fixed-point formats [40], [41]; our bound is the same argument, in ℓ_2 norm and floating point, for the direct form I.